Large Language Models as Realistic Microservice Trace Generators

TraceLLM — LLMs como geradores realistas de traces de microservices

Paulo

Seminário

aclanthology.org/2025.emnlp-main.4

Agenda

1. Contexto e motivação
2. O problema técnico central
3. A solução — TraceLLM
4. Resultados
5. Limitações
6. Conclusão
7. Exercícios

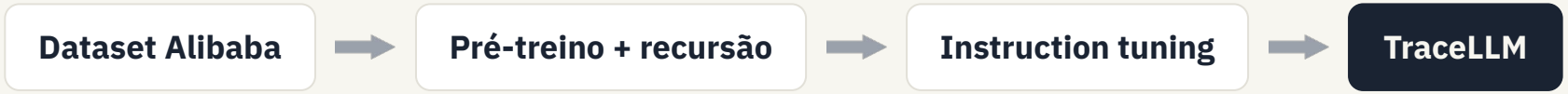
SEÇÃO 2 DE 7

Contexto

O que são traces de microservices, o que é uma LLM e como ela aprende, e o que é fine-tuning com LoRA

Resumo rápido

TraceLLM é um **Llama-2 7B** pré-treinado em traces reais de microserviços (dataset Alibaba) e depois ajustado com duas técnicas novas: geração recursiva camada por camada e instruction tuning com passos intermediários de cálculo — para lidar com a estrutura hierárquica e as restrições desses dados



Traces mais válidos

Muito mais frequentemente do que o baseline tabular

Traces mais realistas

Distribuição próxima dos dados reais de validação

Supera o Llama-3.1 405B

Em tarefas de prever comunicação incomum e preencher dados faltantes

Antes de entrar no artigo

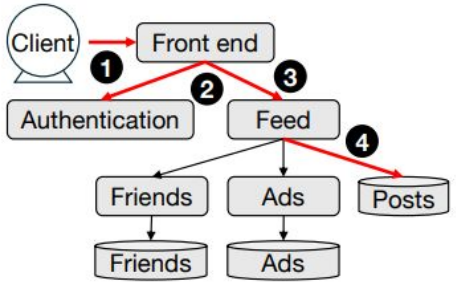
Três conceitos que o artigo usa o tempo todo — um repasse rápido para o resto da apresentação fluir melhor

O que são traces de microservices?

O que é uma LLM e como aprende?

O que é fine-tuning e LoRA?

O que são traces de microservices?



	EDGE ID	Src.	Dest.	Type	Start (ms)	Finish (ms)
1	0	Client	Front end	HTTP	0	24
2	0.1	Front end	Authentication	RPC	0	1
3	0.2	Front end	Feed	RPC	1	23
4	0.2.1	Feed	Posts	DB	6	15

Figura 1 do artigo

O que aconteceu com uma requisição específica, do momento em que entrou no sistema até a resposta final?

- As setas vermelhas são o caminho real de uma requisição: Client → Front end → Authentication / Feed → Posts
- Cada linha da tabela é uma comunicação entre dois microservices — esse é o formato tabular que a LLM precisa aprender a gerar

Como isso é gerado hoje

Microservices: arquitetura comum em apps web, cada serviço pode ser uma aplicação distinta — exige **propagação de contexto** entre eles para montar o trace completo

PYTHON · OPENTELEMETRY

```python
@app.route("/hello/<user_id>")
def hello(user_id):
  with tracer.start_as_current_span("hello") as span:
    span.set_attribute("user.id", user_id)
    return f"Hello, {user_id}!"
```

SPAN GERADO (JSON)

```json
{
  "name": "hello",
  "context": {"trace_id": "5b8aa5…"},
  "parent_id": null,
  "attributes": {"user.id": "42"}
}
```

- O **trace_id** é propagado entre todos os microservices; o **parent_id** dá a estrutura hierárquica. Juntando todos os spans com o mesmo trace_id, temos um trace completo

Traces são um dos pilares da observabilidade

Traces

O caminho de uma requisição entre serviços

Logs

Eventos discretos registrados por cada serviço

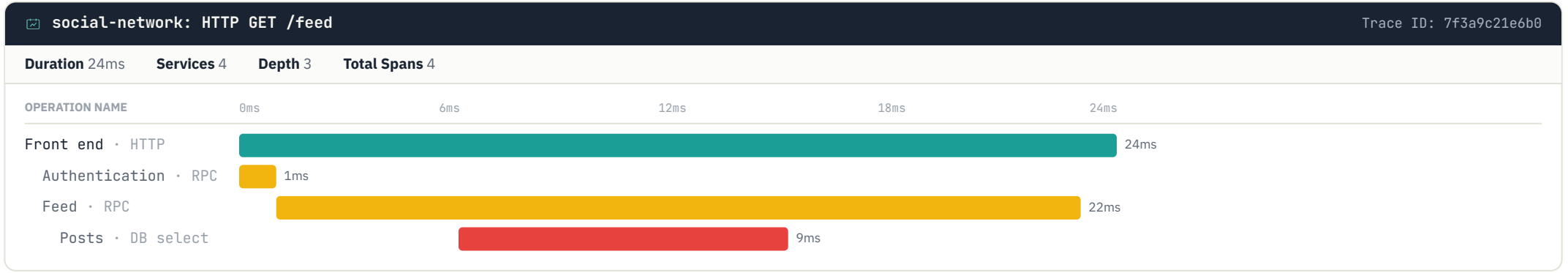
Métricas

Séries numéricas agregadas ao longo do tempo

Profiles

Onde o tempo de CPU é gasto dentro de um processo

Exemplo real: o mesmo trace visto numa ferramenta como Jaeger ou Grafana Tempo

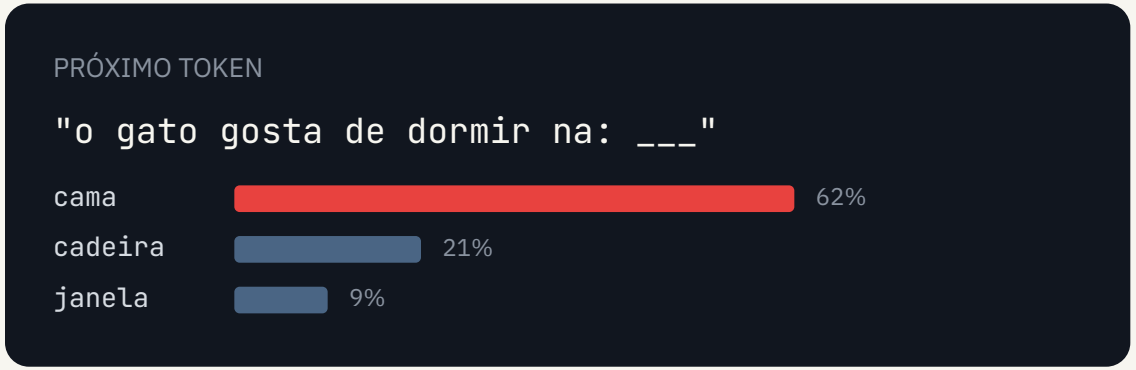


Mesmo trace do slide anterior (edges 0, 0.1, 0.2, 0.2.1) — assim um engenheiro inspeciona onde o tempo foi gasto

O que é uma LLM e como aprende?

Uma **Large Language Model** é uma rede neural treinada para prever o próximo token de um texto

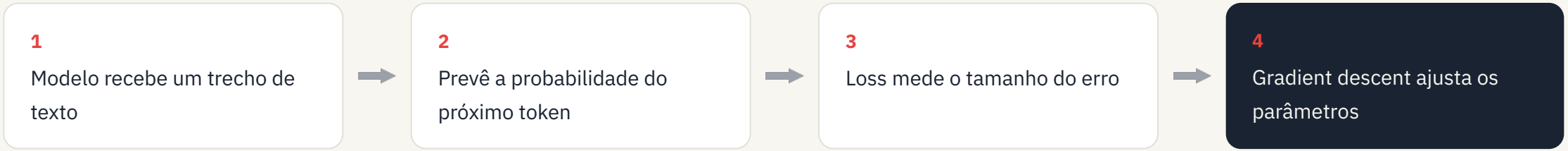
"Large" vem de duas coisas: bilhões de parâmetros ajustáveis, e a grande quantidade de dados de treino. Disso surgem habilidades como resumir texto, responder perguntas e escrever código



Dica: dá pra ver o funcionamento interno de uma LLM em tempo real, token a token, em **poloclub.github.io/transformer-explainer**

O ciclo de treino

Aprendizado auto-supervisionado: o rótulo já está no próprio texto — é o token seguinte



Repete muitas vezes — **steps** (cada ajuste) e **epochs** (cada passada completa pelo dataset) — até o conhecimento ficar guardado nos parâmetros do **transformer**, com o mecanismo de **attention** decidindo quais tokens anteriores importam para prever o próximo

Fine-tuning e LoRA

Fine-tuning continua o treino de um modelo já pronto, num dataset menor e mais específico — aqui, para ele passar a entender traces



- Os parâmetros novos do LoRA são uma fração pequena do tamanho do modelo base
- Usa muito menos recurso computacional
- Reduz o risco de "esquecer" o conhecimento do modelo base

`github.com/ldos-project/TraceLLM/tree/main/checkpoints/llama-2-7b`

SEÇÃO 3 DE 7

Motivação

Por que gerar traces sintéticos, e por que com LLMs?

Por que dados sintéticos de traces

Traces são valiosos

Performance engineering, análise de path crítico — o que de fato atrasa uma requisição

Difícil de ter publicamente

Dados sensíveis — a estrutura de chamadas já revela muito sobre o negócio

Difícil de gerar em escala

Milhares de microservices, times e linguagens diferentes, overhead de performance

Útil para treinar outros modelos

Ex.: detecção de anomalias usando traces sintéticos

A lacuna: trabalhos anteriores geram só **campos específicos** ou estruturas simples — sem lidar com a **hierarquia e as restrições** que um trace de verdade tem

Alguns exemplos de uso real

Critical path analysis

Uber — CRISP, análise de caminho crítico em arquiteturas de microservices

uber.com/.../crisp-critical-path-analysis

Performance engineering

AWS — identificar e resolver problemas de performance com traces

aws.amazon.com/.../analyze-aws-microservices-architecture

Detecção de anomalias

New Relic — distributed tracing aplicado à detecção de anomalias

newrelic.com/.../distributed-tracing-anomaly-detection

Por que explorar LLMs para isso?

01 LLMs já foram adaptadas para domínios específicos como geração de código — por que não traces?

02 LLMs seguem instruções e generalizam para prompts novos — dá pra pedir traces de cenários variados em linguagem natural

SEÇÃO 4 DE 7

O problema técnico central

Como gerar traces consistentes com um modelo que escreve token por token e nunca volta atrás?

Um trace é um DAG com restrições

⚠ LLMs são **autorregressivos**: geram um token por vez e não voltam para corrigir. Uma condição errada em um nó pode contaminar o resto do trace

É um grafo acíclico direto

Sem ciclos — uma requisição sempre caminha para o futuro, cada chamada ocorre num timestamp diferente

Tempo precisa ser respeitado

Um filho não pode durar mais que o pai — se o trace todo dura 10ms, nenhum filho pode durar 20ms

Condições do trace

Nº de nós e profundidade pedidos precisam ser exatamente respeitados na geração

Validações detalhadas no artigo, apêndice "Constraints in Call Graph Layers"

SEÇÃO 5 DE 7

A solução — TraceLLM

github.com/ldos-project/TraceLLM

Dataset — Alibaba v2022

Traces reais, anonimizados, públicos — primeira hora de dados de mais de 10 clusters

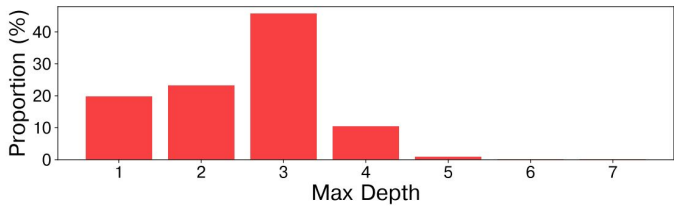
`github.com/alibaba/clusterdata/.../cluster-trace-microservices-v2022`

6 434
microservices únicos

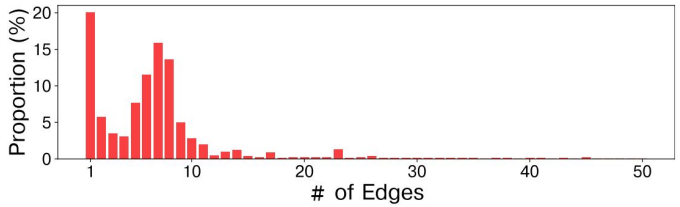
1,36 M
traces de microservices

1,1 B
tokens de treino

10%
separado para validação



(a) Distribution by call graph depth.



(b) Distribution by the number of edges.

- Service ID começa com **S_**, microservice com **MS_** — um service agrupa vários microservices
- Traces construídos agrupando api calls pelo mesmo trace id
- Missing values removidos — outro artigo cita 67% dos traces da Alibaba com algum valor faltando

Training setup

Llama-2 7B
modelo base

4× A100 80GB
hardware de treino

T = 0,8 · Top K = 50
inferência (default)

3 adapters LoRA, mesmo Llama-2 7B base

baseline
Formato tabular (GReaT), sem recursão

recursive
Geração recursiva, camada por camada

recursive + instruction
Recursão + instruction tuning

Também comparam com outros tamanhos de modelo: Llama-3.2 1B / 3B, Llama-2 13B, e Llama-3.1 405B

Encoding — de tabela para texto

Técnica **GReaT** (Generation of Realistic Tabular Data)

arxiv.org/abs/2210.06280

TABULAR

```
Edge Id = 0
```



TEXTO NATURAL

```
Edge Id is 0
```

</>

Texto puro

LLMs conhecem linguagem natural, e cada valor já carrega seu rótulo junto — sem depender de posição de coluna

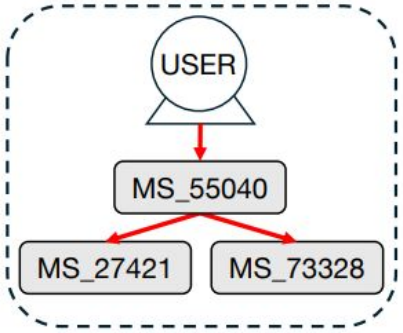
Shuffle dos campos

Em dois níveis: campos de cada edge (ex: posição do Edge Id) e condições globais do trace — o modelo aprende o significado, não a posição

Drop de condições

Cada condição global tem 90% de chance de sumir do treino — flexibiliza o prompt do usuário

Exemplo — baseline (GReaT), pré-treino




```
[GENERATE GRAPH]
num_edges:3/id:S_058367691/max_depth:2
<edges>
(source is USER,communication finishes at 2 milliseconds,communication starts at 0 milliseconds,type is rpc,destination is MS_55040,edge_id is 0)
(communication starts at 1 milliseconds, source is MS_55040,communication finishes at 1 milliseconds,destination is MS_27421,edge_id is 0.1,type is db)
(communication finishes at 2 milliseconds,type is db,communication starts at 1 milliseconds,destination is MS_73328, source is MS_55040 ,edge_id is 0.2)
</edges>
```

- **num_edges:3 / max_depth:2** — as condições globais, embaralhadas, iguais ao que o usuário poderia pedir
- Cada linha dentro de <edges> é um api call — source, destination, tipo e tempo, em texto

Recursive Generation

Gerar um trace de 40 nós e profundidade 3 de uma vez é difícil — qualquer erro invalida o trace inteiro. A ideia: quebrar em **tarefas pequenas e recursivas**, inspirado em *Tree of Thoughts*

tarefa pequena + **tarefa pequena** + **tarefa pequena** = **trace completo**

USO 1

No treinamento

Formatação dos dados: o trace é quebrado em camadas (layers) para o fine-tuning

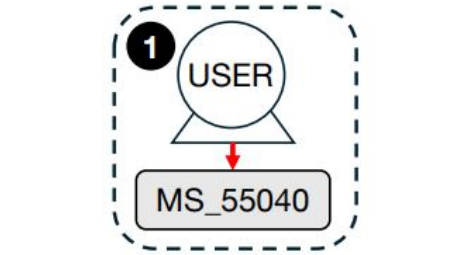
USO 2

Na inferência

O modelo é chamado várias vezes, uma camada por vez, até completar o trace

Inspirado em Tree of Thoughts — arxiv.org/abs/2305.10601

Recursive Generation — no treinamento



1
[GENERATE GRAPH]
num_current_edges:1/num_subgraphs:1/num_edges:3/start_communication_at:0/id:S_058367691/remaining_depth:1/
start_edge_id:0/start_node:USER
<layer>
<edges>
(communication finishes at 2 milliseconds,communication starts at 0 milliseconds,type is rpc,destination is MS_55040,edge_id is 0)
</edges>
<subgraph of edge_id 0>
latency:2/start_node:MS_55040/start_edge_id:1/num_subgraphs:0/num_current_edges:2/id:S_058367691/num_edges:2/
remaining_depth:0/start_communication_at:1/caller:USER
</subgraph>
</layer>

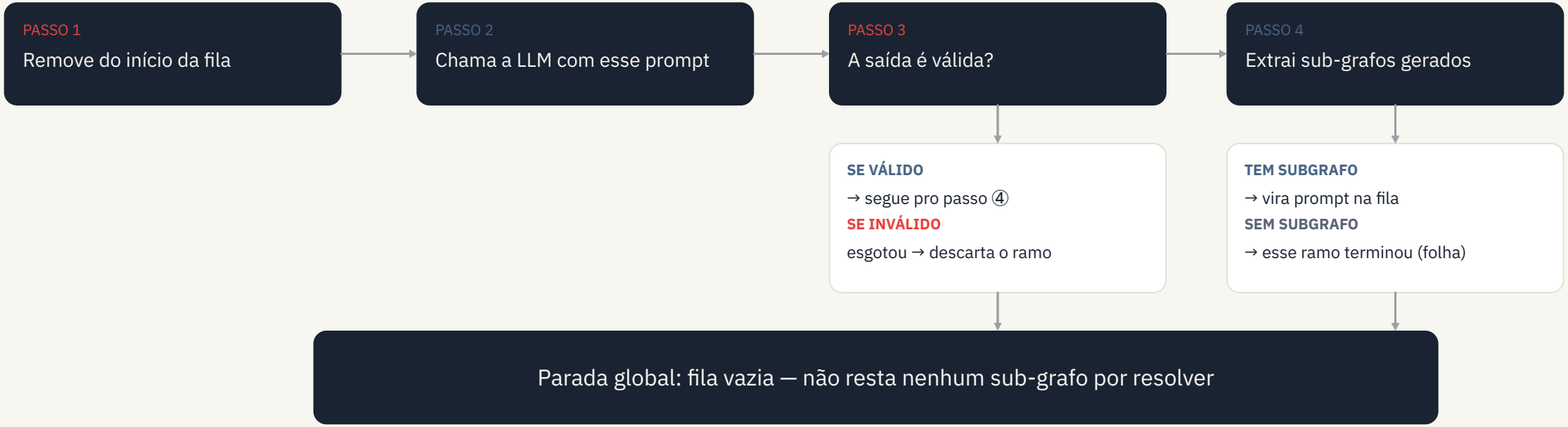
2
start_edge_id:1/id:S_058367691/latency:2/num_subgraphs:0/num_edges:2/remaining_depth:0/start_communication_at:1/
num_current_edges:2/start_node:MS_55040/caller:USER
<layer>
<edges>
(communication starts at 1 milliseconds,communication finishes at 1 milliseconds,destination is MS_27421,edge_id is 1,type is db)
(communication finishes at 2 milliseconds,type is db,communication starts at 1 milliseconds,destination is MS_73328,edge_id is 2)
</edges>
</layer>

① **Layer 1** — a chamada USER → MS_55040, com as condições globais e uma tag **<subgraph of edge_id 0>** com a condição pra próxima layer

② **Layer 2** — gerada a partir dessa condição, contém as duas chamadas seguintes de MS_55040

Recursive Generation — na inferência

Pseudocódigo próprio, a partir do código do TraceLLM — o artigo não detalha essa parte



Fila = deque: consumida do início (FIFO), mas filhos e retries entram **no início**, furando a fila — na prática o comportamento é mais LIFO/profundidade-primeiro. **sub_chamada** = o marcador <subgraph of edge_id N> que o próprio modelo escreveu

Caso 1 — condições prontas: usuário já informa num_edges/depth antes de entrar na fila

Caso 2 — prompt genérico: ex. "alta latência" — a LLM gera as condições na 1ª layer, e elas são extraídas pro loop

Instruction Tuning

Um fine-tuning a mais, em cima do modelo **recursive** — 5% dos grafos de treino

Melhorar a geração recursiva

O modelo às vezes violava restrições entre camadas — ex: latência de uma layer maior que a do trace todo

Flexibilizar o prompt

Permitir pedidos em linguagem natural, tipo "gera um trace com alta latência"

Instruções intermediárias: passos de raciocínio em linguagem natural antes de cada camada — inspirado em Chain-of-Thought (Wei et al., 2022) e Scratchpads (Nye et al., 2021). As condições iniciais do prompt são excluídas do loss

Exemplo — recursive + instruction

Instruction

```
You are a trace generator that creates traces based on given requirements.
Requirements:
start_communication_at:0/start_node:USER/remaining_depth:2/num_current_edges:1/num_edges:4/latency:12/id:S_032647104
Conditions:
In each edge, communication start time should NOT be greater than latency 12 milliseconds
Generate subgraph instructions if necessary
the first start_communication_at should be requirement's start_communication_at 0
Also, communication should finish before latency 12 milliseconds
copy caller from requirement's start_node:USER
generate 1 edges following num_current_edges
```

Output

```
<layer>
<edges>
(edge_id is 0,type is http,communication starts at 0 milliseconds,destination is MS_57649,communication finishes at 12 milliseconds)
</edges>
num generated edges = the last edge id - the first edge id + 1 = 0 - 0 + 1 = 1
1 edges generated out of num_edges:4
num_remaining_edges = num_edges:4 - generated:1 = 3

generate subgraphs of edge:0
Subgraph constraints:
remaining_depth should be the requirement's remaining_depth:2 - 1 = 1
num_edges <= num_remaining_edges:3
copy start_node from edge 0 destination: MS_57649
<subgraph of edge_id 0>
remaining_depth:1/start_edge_id:1/num_edges:3/id:S_032647104/latency:12/num_subgraphs:1/num_current_edges:2/
start_node:MS_57649/start_communication_at:1/caller:USER
</subgraph>
now, num_remaining_edges is 3 - 3 = 0
finish generation
</layer>
```

- **Instruction** — concatenado com o Output, é o texto completo que a LLM aprende; é aqui que entram coisas como a latência que ela precisa respeitar, dando mais rigidez ao que foi pedido no trace
- **Output** — os rascunhos de raciocínio aparecem em dois pontos: logo após </edges> e antes de cada <subgraph> — a formatação em layers da geração recursiva continua presente

Os três formatos, no repositório

baseline

`dataset_examples/`
`tabular_dataset.txt`

recursive

`dataset_examples/`
`recursive_dataset.txt`

recursive + instruction

`dataset_examples/`
`recursive_instruction_dataset.txt`

SEÇÃO 6 DE 7

Resultados

Structured Reasoning Accuracy

Structured Reasoning Accuracy

Acurácia dos 3 modelos — baseline, recursive e recursive + instruction — variando **num_edges** (1–30), **depth** (1–6) e **temperatura**. 50 amostras por combinação

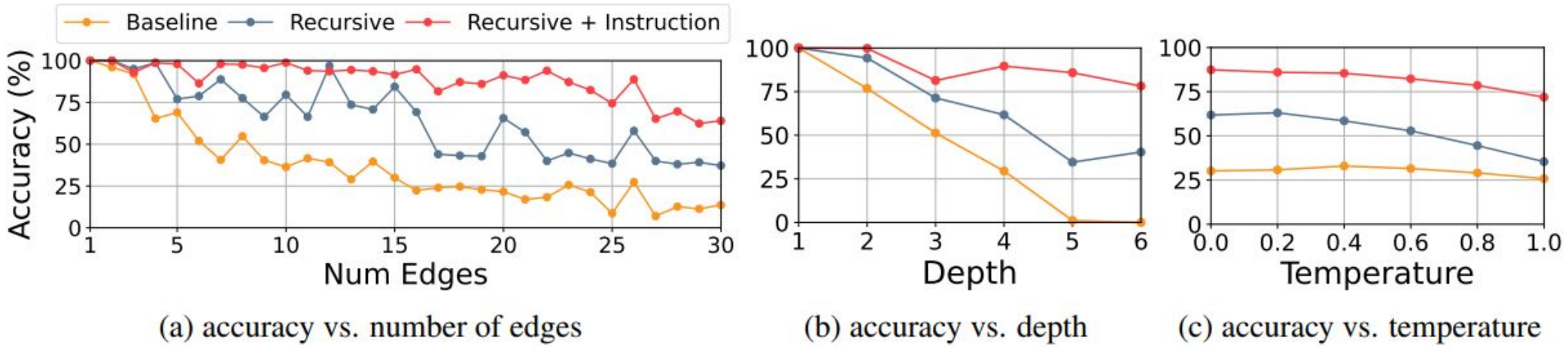
<25%
baseline, acurácia com mais de 15 edges

~0%
baseline, acurácia com depth maior que 4

mantém alta
recursive + instruction, mesmo com mais edges e mais depth

Atribuído a quebrar a geração em pedaços pequenos e gerenciáveis — em vez de um trace inteiro de uma vez

Figura 3 — accuracy por edges, depth e temperatura



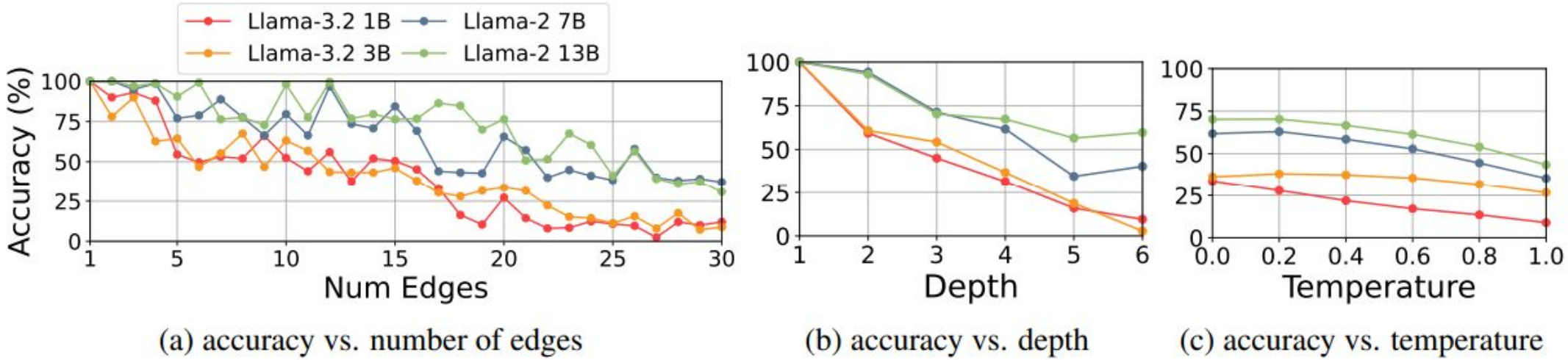
(a) accuracy vs. number of edges (b) accuracy vs. depth (c) accuracy vs. temperature

O **recursive + instruction** mantém acurácia alta mesmo em grafos grandes e profundos — a diferença pro baseline cresce exatamente onde a tarefa fica mais complexa. Greedy decoding em (a)/(b) (ou seja, temperatura 0)

O artigo reporta que o **recursive** (sem instruction) já mantém acurácia ~30% maior que o baseline em (a), e ~35% maior em (b). Em (c), o **recursive** mantém ~10% a mais que o baseline mesmo em temperatura 1, e o **instruction tuning** melhora a acurácia em 23% a 36%

O tamanho do modelo também importa

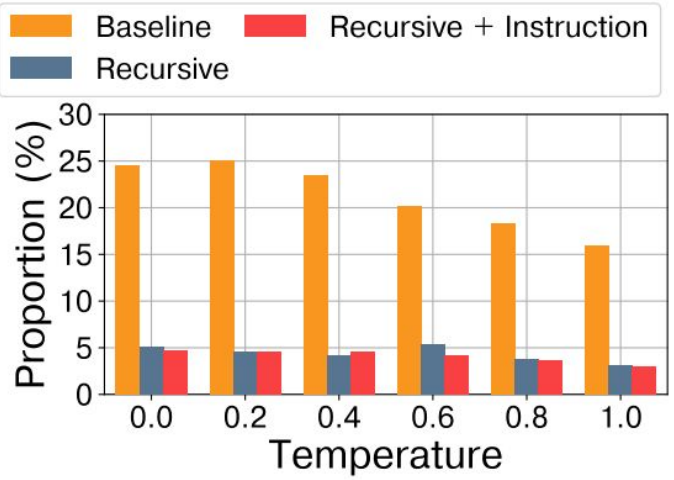
Apêndice D.2 — mesma técnica recursive, variando o modelo: Llama-3.2 1B / 3B, Llama-2 7B, Llama-2 13B



O **Llama-2 13B** fica bem mais consistente que o 7B, principalmente com depth maior — mais parâmetros, mesmo fine-tuning, acurácia mais alta. Isso sugere que aplicar esse mesmo fine-tuning num modelo ainda maior pode render uma acurácia melhor

O modelo memorizou, ou aprendeu o padrão?

Apêndice D.3 — % de traces sintéticos que batem exatamente com algum trace do treino



16–24%

baseline — memoriza bastante

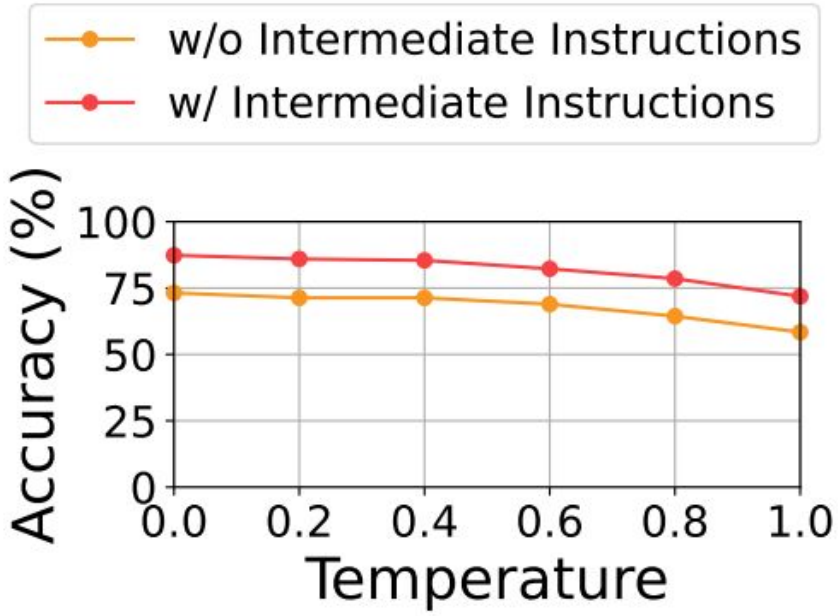
3–5%

recursive e recursive + instruction

A geração recursiva não só gera traces mais válidos — também gera traces mais **diversos**, menos copiados do treino

As instruções intermediárias ajudam mesmo?

Apêndice D.1 — ablação: com vs. sem os passos de raciocínio no instruction tuning



13% de queda na acurácia ao remover as instruções intermediárias — em todas as temperaturas

SEÇÃO 6 DE 7 · RESULTADOS

Similarity of Real and Synthetic Traces

Os traces sintéticos realmente se parecem com os reais?

Comparando com os traces reais de validação

O modelo testado aqui é o **recursive + instruction** ("Ours") — e como os baselines não geram call graphs completos como o TraceLLM, a comparação é por **propriedades/distribuições** dos traces, não pelo trace inteiro

Baselines

GReaT — é o mesmo modelo **baseline**: Llama-2 7B tabular, treinado no mesmo dataset

Probabilistic model — gerador estatístico da Alibaba

TVAE — VAE para dados tabulares

Cenário 1 — chamadas populares

Distribuição de (origem, destino, tipo) — medida com **KL divergence** (0 = idênticas)

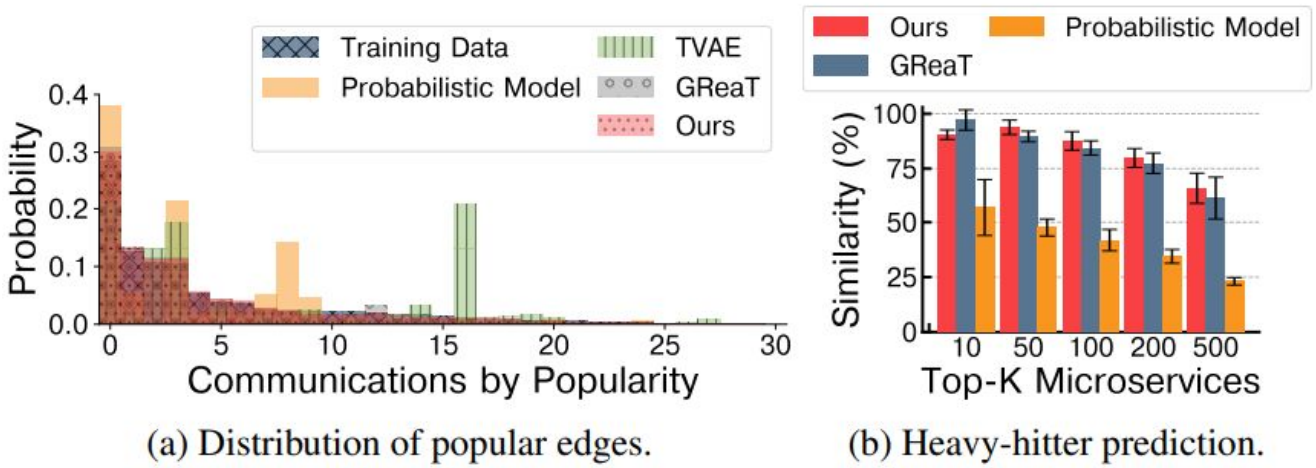
Ex: se MS A chama MS B com um tipo específico com frequência no real, isso deve se repetir no sintético

Cenário 2 — heavy hitters

Top-K microservices mais chamados (K de 10 a 500) — real vs. sintético

Ex: um microservice de autenticação, chamado por muitos outros, é um heavy hitter

Figura 4 — distribuição de chamadas e heavy-hitters



CENÁRIO A — chamadas populares (a)

Modelo	KL
Ours (recursive+instruction)	0,16
GReaT	0,11
TVAE	0,74
Probabilistic model	3,84

KL menor = mais parecido com o real. No gráfico ao lado, o Probabilistic model tem picos nas chamadas de rank ~5–10 — dá mais atenção a elas do que existe de fato no dataset de treino

CENÁRIO B — heavy hitters (b)

Ours fica acima de 90% de similaridade pra K≤50 e 65% em K=500 — bem acima do probabilistic model. Quanto maior o K, mais microservices o modelo precisa acertar como populares, por isso a similaridade cai

SEÇÃO 6 DE 7 · RESULTADOS

Synthetic Data as Training Data

Os traces sintéticos servem para treinar outros modelos de gerenciamento de microservices?

Dois cenários de gerenciamento de microservices

Critical Component Extraction

FIRM (SVM) prevê microservices com risco de violar SLO de latência — treinado com dados reais vs. sintéticos

Anomaly Detection

TraceVAE detecta microservices com latência fora do comum — real vs. sintético, medido em ROC AUC

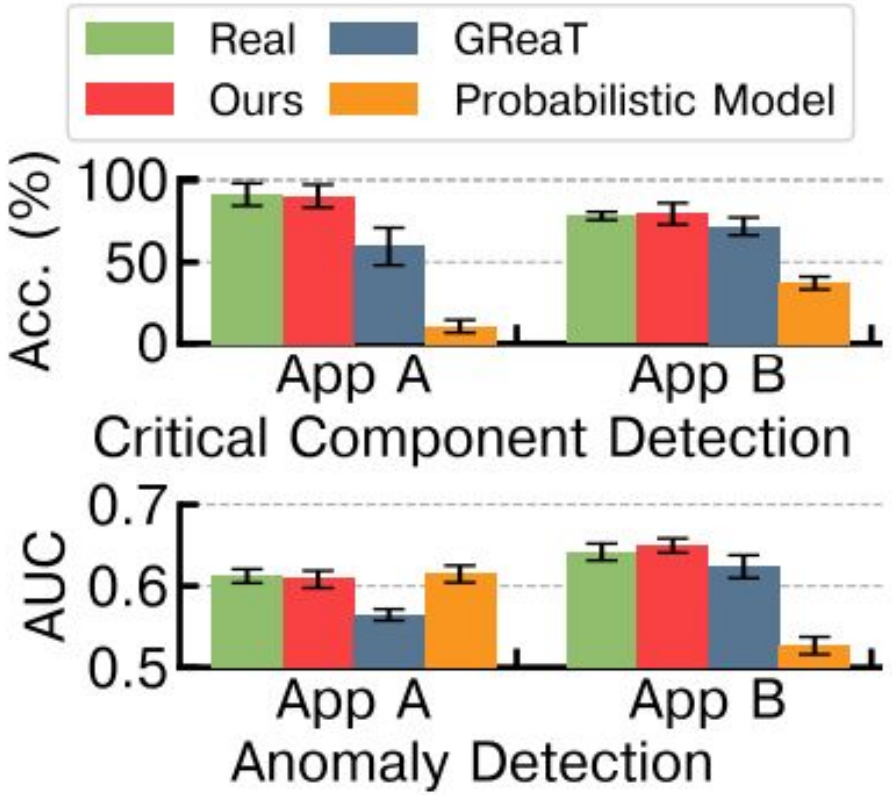
Traces reais → **Extrai conditions** → **Gera traces sintéticos** → **Treina FIRM / TraceVAE**

Os traces sintéticos vêm de 3 geradores: **recursive + instruction**, **GReaT (baseline)** e **probabilistic model**

Testado em apenas 2 services populares (App A e App B), cada um com seu próprio conjunto de microservices — não é uma amostra ampla de services

5K traces sintéticos para treino, testado em 2K traces reais — mesmo conjunto de teste em todos os experimentos

Figura 5 — desempenho treinado com dados reais vs. sintéticos



CRITICAL COMPONENT DETECTION

Modelos treinados com dados do **Ours** ficam a menos de **1,5 ponto percentual** do desempenho com dados reais — GReaT e Probabilistic Model ficam de 6 a 81 pontos abaixo

ANOMALY DETECTION

Cenário parecido, medido em AUC. Exceção: na App A o Probabilistic Model teve AUC mais alto que o Ours, mas foi bem pior na App B — provável acaso

SEÇÃO 6 DE 7 · RESULTADOS

Instruction-following Capability

O modelo realmente obedece instruções específicas de atributo?

Duas instruções — nunca combinadas no treino

Treinadas de forma isolada na etapa de **instruction tuning**

High Latency

Latência acima do p90 do serviço

`"Build a call graph with high latency"`

Uncommon Communications

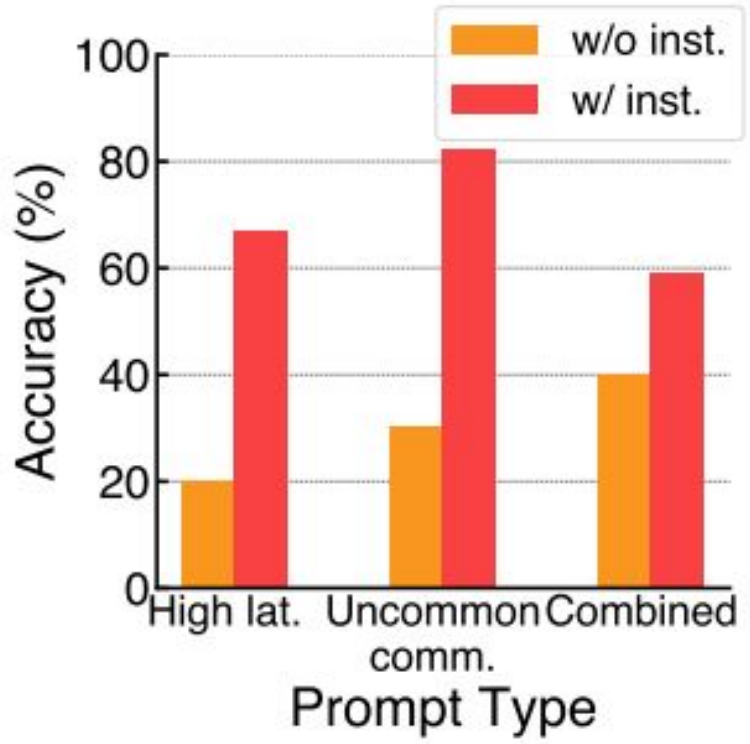
Chamada que ocorre em menos de 10% dos dados

`"Include an edge from (SRC) to (DEST)..."`

Essas duas instruções **nunca aparecem juntas** no treino — testar a combinação mostra se o modelo generaliza, ou só memorizou cada uma isolada.

1K prompts de validação · comparado com/sem instrução (w/ inst. vs. w/o inst.)

Figura 6 — accuracy de instruction-following



W/ INST. VS. W/O INST.

w/o inst. = o modelo gerou aquele atributo por acaso, sem ter sido pedido — natural, já que com tantos microservices testados, alguns já têm naturalmente latência alta ou comunicação rara. Mesmo assim, a diferença pro cenário com instrução é grande

COMBINADO

Cenário nunca visto no treino — a acurácia não só fica acima do acaso, ela se mantém tão alta quanto nos cenários individuais: generaliza sem perder desempenho

SEÇÃO 6 DE 7 · RESULTADOS

Adapting Models for Downstream Tasks

O conhecimento sobre traces serve pra outras tarefas, além de gerar traces?

Três modelos, duas tarefas de classificação

Llama-3.1 405B

Sem fine-tuning — 16 exemplos no prompt (in-context)

Llama-2 7B cru

Fine-tuning só para classificação, sem pré-treino em traces

Ours (recursive+instr.)

Já pré-treinado em traces + fine-tuning para classificação

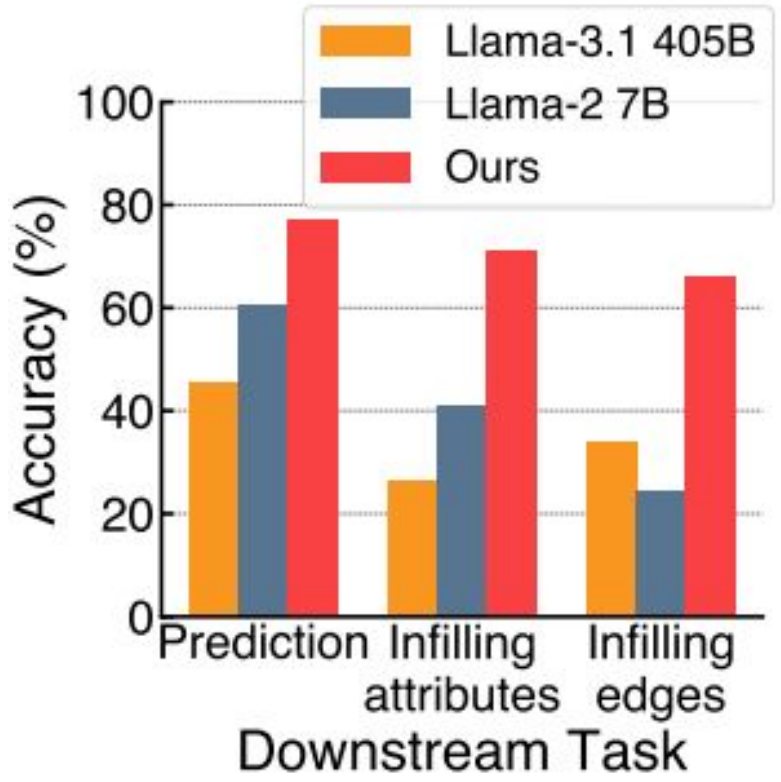
Binário — recebe 10 linhas do trace e diz se a próxima tem comunicação rara (<10% do dataset) ou não

15K samples de treino

Multi-classe — preenche um atributo (tipo ou destino) ou um edge inteiro que falta (infilling)

Atributo: 1,2K treino / 6K validação · Edge completo: 1K treino / 5K validação

Figura 7 — accuracy nas tarefas downstream



OURS NA FRENTE

Prediction = binário (comunicação rara); **Infilling attributes** e **infilling edges** = as duas tarefas multi-classe. O Ours fica com acurácia alta nas três, acima dos outros dois modelos

405B FICA ATRÁS DO 7B

Mesmo sendo bem maior, o 405B não teve fine-tuning (só 16 exemplos no prompt) — modelos generalistas grandes têm dificuldade em domínio específico que não viram no treino; o pré-treino em traces é o que faz diferença

SEÇÃO 7 DE 7

Limitações

O que os próprios autores apontam como pontos em aberto

Três limitações que o artigo assume

O histórico é descartado a cada camada

Só as condições da camada atual seguem adiante — bom pra manter o prompt pequeno, mas perde cenários onde o histórico importaria (cache esquentando, etc.). Comprimir sem perder informação relevante é um problema em aberto

Templates de instrução feitos à mão

Poderiam usar o poder de LLMs maiores para gerar variações — mas isso envolve checar se o resultado ainda tem as características certas de um trace

Testado só em traces de microservices

A ideia de dado hierárquico com restrições deveria valer para outros tipos de trace — ainda por explorar (ver apêndice D.7, batch jobs)

SEÇÃO 7 DE 7

Conclusão

O **TraceLLM** mostra que uma LLM pode gerar traces de microservices **válidos e realistas** — e que esses traces sintéticos substituem dados reais no treino de outros modelos de gerenciamento de microservices

- Geração recursiva + instruções intermediárias melhoram muito a validade estrutural
- A técnica pode se estender a outros traces hierárquicos — não só microservices
- Fine-tuning de domínio importa — até o Llama-3.1 405B perde pro 7B especializado
- Modelos maiores, com o mesmo fine-tuning, tendem a uma acurácia ainda melhor

SEÇÃO 7 DE 7

Exercícios

Um notebook para consolidar o que vimos hoje

3 exercícios, cada um com questões (a) e (b)

EXERCÍCIO 1

Accuracy por tamanho

Reproduz a Figura 3a/3b — accuracy por num_edges e por depth

EXERCÍCIO 2

Accuracy por temperatura

Reproduz a Figura 3c — accuracy vs. temperatura, nos 3 modelos: baseline, recursive e recursive + instruction

EXERCÍCIO 3

Instruction-following

Reproduz a Figura 6 — o modelo obedece instruções combinadas?

- Llama-2 7B real + os adapters LoRA publicados no GitHub — inferência de verdade, gerando traces sintéticos
- Não precisa executar — olhem o código, os gráficos já gerados, a pergunta e o artigo, e respondam
- Rodei numa RTX 5070 12GB — levou algumas horas; na T4 do Colab pode demorar também
- Notebook gerado com Claude olhando o repo do TraceLLM — não usa vLLM, mas usa trechos do próprio repo, tentando reproduzir fielmente o repo e o artigo